BMS COLLEGE OF ENGINEERING
Bengaluru -560019

Autonomous College, Affiliated to
Visvesvaraya Technology University, Belgaum

Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)
Course Code: 23ECSPE1AD

Submitted By
MITHUN V - 1BM24EC111

Submitted To
Prof. AJAYKUMAR DEVARAPALLI

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year
2026 – 2027

TABLE OF CONTENT

SL.NO	TITLE	PAGE NO.
1	Arithmetic logic unit	1
2	Comparator	3
3	4 – bit parity generator and checker	5
4	4 – bit barrel shifter	7
5	Timer module	9
6	Ring counter	11
7	Johnson counter	13
8	Synchronous and asynchronous counters	15
9	Shift counter	19
10	Divide by N counter	21
11	Mealy and Moore FSM	23
12	Random sequence generator	27

1. ARITHMETIC LOGIC UNIT

TOOL USED: VIVADO (XILINX)

CODE:

Design block

```
module alu #(parameter n=8)(
    input  wire [n-1:0] A, // First Operand (e.g., 8 bits: 7:0)
    input  wire [n-1:0] B, // Second Operand (e.g., 8 bits: 7:0)
    input  wire [2:0] sel, // Operation Selector Opcode
    output reg  [n:0] out ); // Output scaled to n:0 (e.g., 9 bits: 8:0)

    always @(*) begin
        out = {(n+1){1'b0}};// Default assignment to prevent accidental latch synthesis
        case(sel)
            3'b000: out = A + B;// Addition (Carry automatically goes to out[n])
            3'b001: out = A - B;// Subtraction (Borrow/Carry goes to out[n])
            3'b010: out = {1'b0, (A & B)}; // Bitwise AND
            3'b011: out = {1'b0, (A | B)};// Bitwise OR
            3'b100: out = {1'b0, (A ^ B)};// Bitwise XOR
            3'b101: out = {1'b0, (A << 1)};// Logical Shift Left
            3'b110: out = {1'b0, (A >> 1)};// Logical Shift Right
            3'b111: out = (A == B);// Equal Comparison
            default: out = {(n+1){1'b0}};
        endcase
    end
endmodule
```

Test bench

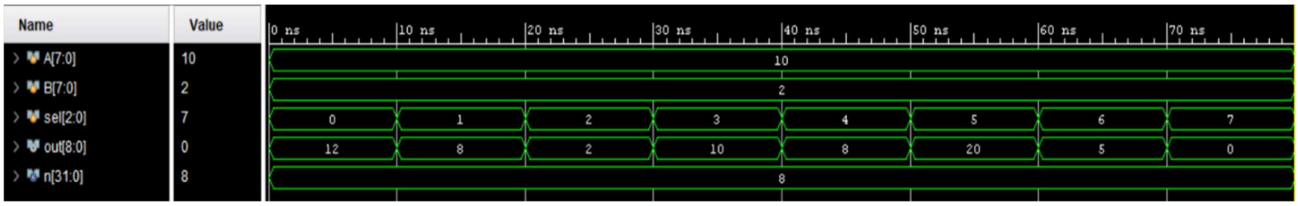
```
`timescale 1ns / 1ps

module tb_alu;
parameter n=8;
reg [n-1:0]A;
reg [n-1:0]B;
reg [2:0]sel;
wire [n:0] out;
alu A1 (A,B,sel,out);
initial begin
$monitor ("Time=%0t | Sel=%b | A=%d, B=%d | Out=%d",$time, sel, A, B, out);
A=8'h0A;B=8'h02;sel=3'b000;#10
sel=3'b001;#10
sel=3'b010;#10
sel=3'b011;#10
sel=3'b100;#10
sel=3'b101;#10
sel=3'b110;#10
sel=3'b111;#10
$finish;
end
endmodule
```

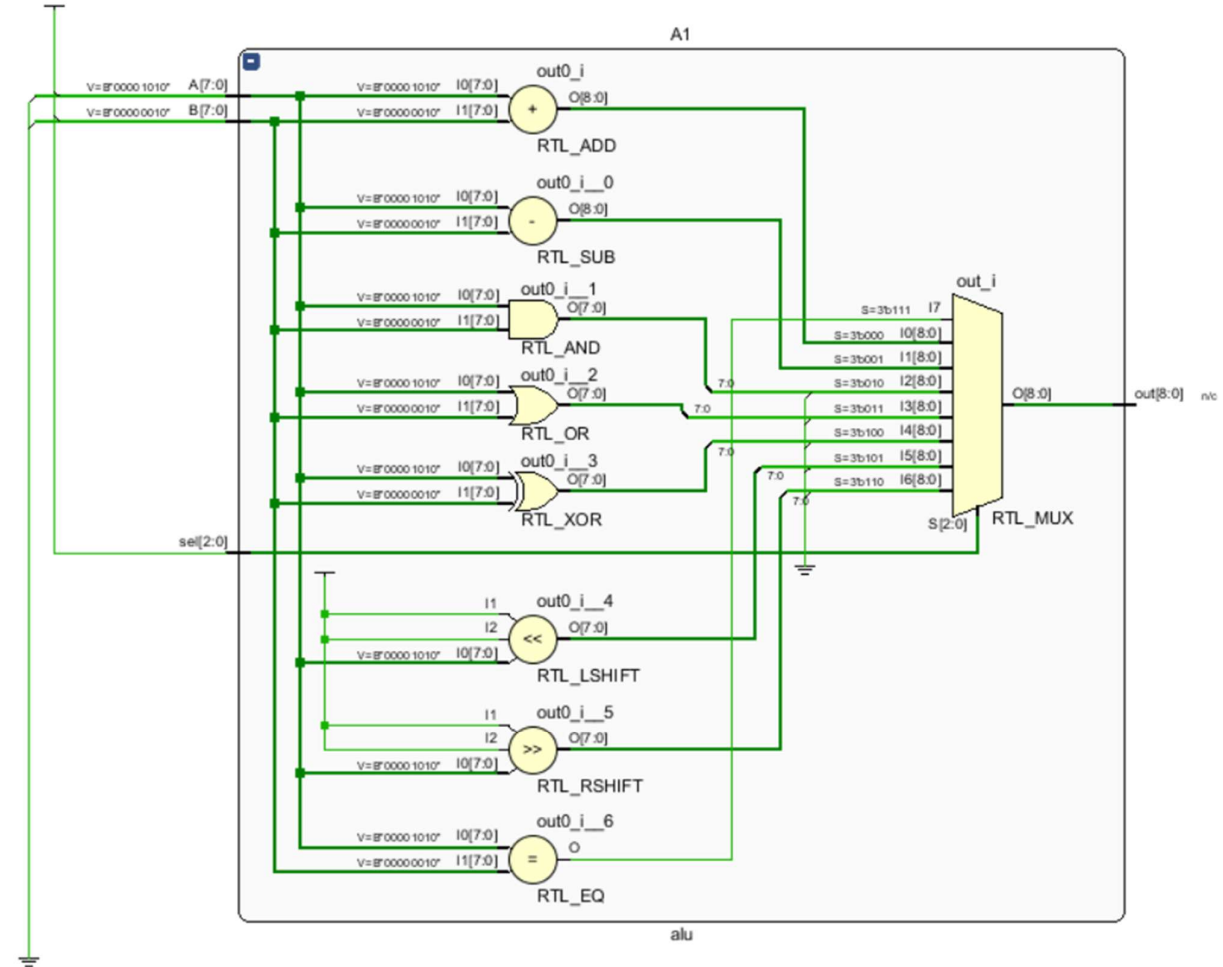
OUTPUT:

```
# run 1000ns
Time=0 | Sel=000 | A= 10, B=  2 | Out= 12
Time=10000 | Sel=001 | A= 10, B=  2 | Out=  8
Time=20000 | Sel=010 | A= 10, B=  2 | Out=  2
Time=30000 | Sel=011 | A= 10, B=  2 | Out= 10
Time=40000 | Sel=100 | A= 10, B=  2 | Out=  8
Time=50000 | Sel=101 | A= 10, B=  2 | Out= 20
Time=60000 | Sel=110 | A= 10, B=  2 | Out=  5
Time=70000 | Sel=111 | A= 10, B=  2 | Out=  0
```

WAVEFORM:



SCHEMATIC DIAGRAM:



2. COMPARATOR

TOOL USED: VIVADO (XILINX).

CODE:

Design block

```
module comp #(parameter n=8)(
input [n-1:0] a,b,
output reg agb,aeb,alb);
always @ (*) begin
if (a>b) begin
agb=1;aeb=0;alb=0;
end
else if (a==b) begin
agb=0;aeb=1;alb=0;
end
else if (a<b) begin
agb=0;aeb=0;alb=1;
end
end
endmodule
```

Test bench

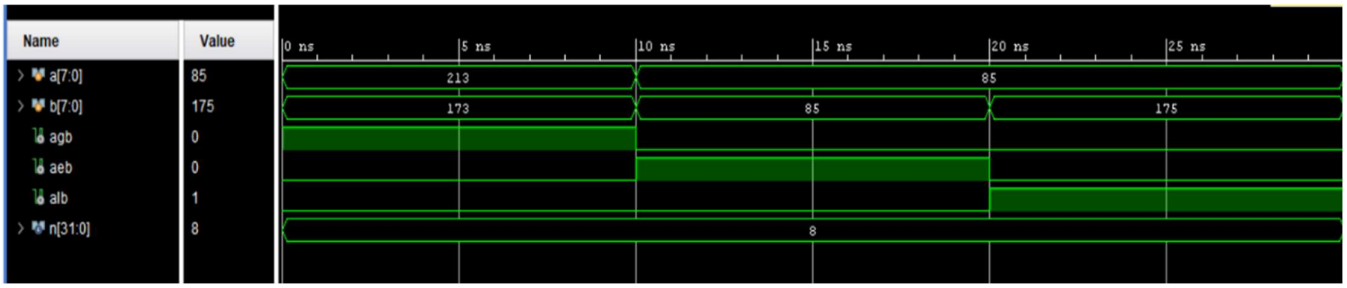
```
`timescale 1ns / 1ps

module tb_comp;
parameter n=8;
reg [n-1:0]a,b;
wire agb,aeb,alb;
comp C1 (a,b,agb,aeb,alb);
initial begin
$monitor("Time=%0t | A=%d | B=%d, A>B =%b | A=B =%d, A<B =%b",$time, a, b, agb, aeb, alb);
a=8'b11010101;b=8'b10101101;#10
a=8'b01010101;b=8'b01010101;#10
a=8'b01010101;b=8'b10101111;#10
$finish;
end
endmodule
```

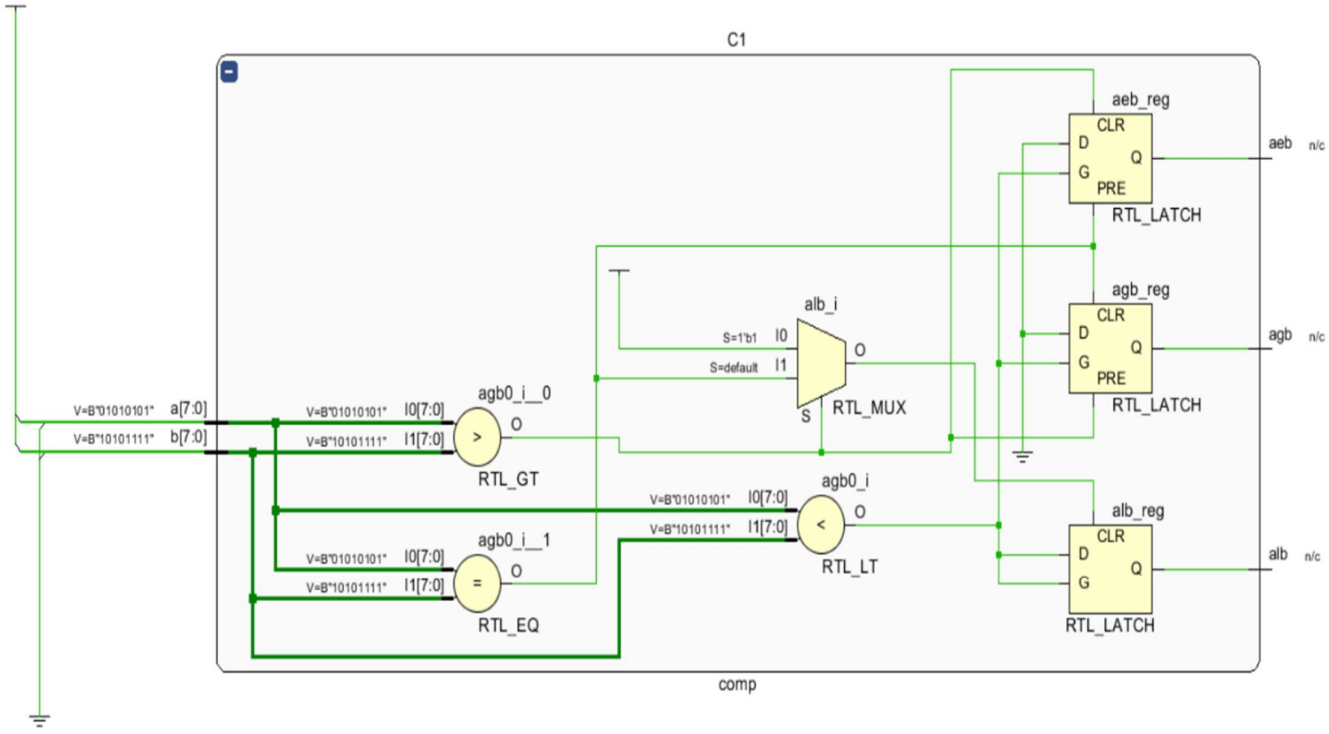
OUTPUT:

```
# run 1000ns
Time=0 | A=213 | B=173, A>B =1 | A=B =0, A<B =0
Time=10000 | A= 85 | B= 85, A>B =0 | A=B =1, A<B =0
Time=20000 | A= 85 | B=175, A>B =0 | A=B =0, A<B =1
```

WAVEFORM:



SCHEMATIC DIAGRAM:



3. 4 – BIT PARITY GENERATOR AND CHECKER

TOOL USED: VIVADO (XILINX).

CODE:

Design block

```
module parity(
//transmittor
input [3:0]tx_data,
output tx_parity,
//reciever
input [3:0]rx_data,
input rx_parity,
output error
    );
    assign tx_parity = ^(tx_data);
    assign error= (^rx_data)^rx_parity;
endmodule
```

Test bench

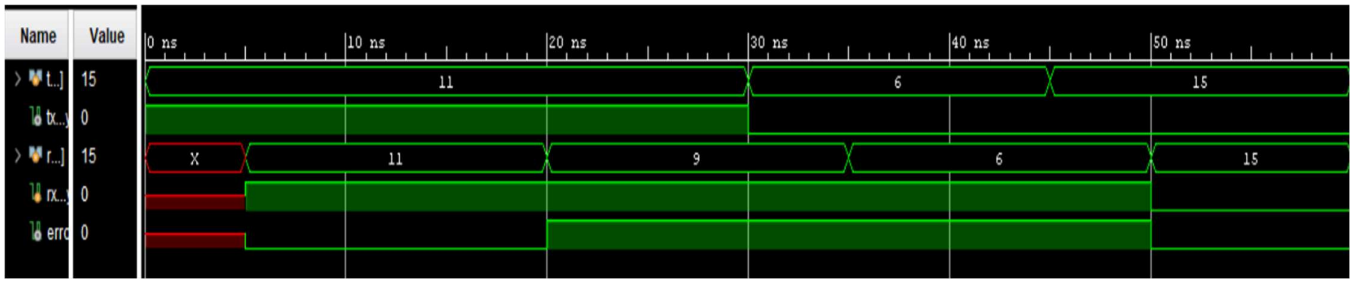
```
module tb_parity;
reg [3:0] tx_data;
wire tx_parity
reg [3:0] rx
reg rx_parit
wire error;
parity P1 (tx_data,tx_parity,rx_data,rx_parity,error);
initial begin
$monitor("%0t\t%b\t%b\t%b\t%b\t%b",
                $time, tx_data, tx_parity, rx_data, rx_parity, error);
// Case 1: Clean Transmission (tx_data = 1011 -> 3 ones, tx_p = 1)
tx_data = 4'b1011; #5;
rx_data   = tx_data;
rx_parity = tx_parity;
#10;

//Case 2: Data Bit Flip Error (rx_data becomes 1001, tx_p remains 1)
tx_data = 4'b1011; #5;
rx_data   = 4'b1001; // Bit 1 flipped!
rx_parity = tx_parity;
#10;

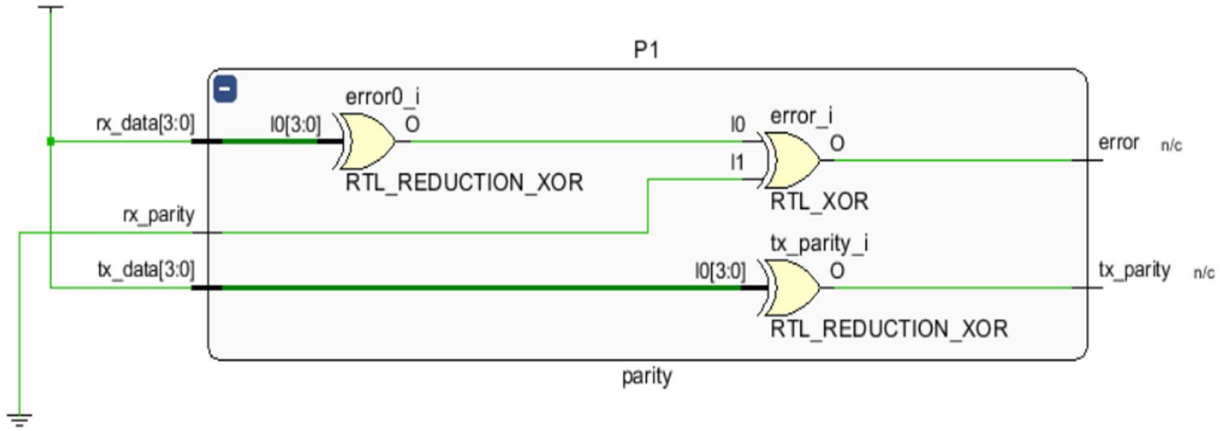
//Case 3: Parity Bit Flip Error (rx_data intact, rx_parity corrupted)
tx_data = 4'b0110; #5;
rx_data   = tx_data;
rx_parity = ~tx_parity; // Parity bit flipped!
#10;

//Case 4: Clean Transmission (tx_data = 1111 -> 4 ones, tx_p = 0)
tx_data = 4'b1111; #5;
rx_data   = tx_data;
rx_parity = tx_parity;
#10;
$finish;
end
```

OUTPUT WAVEFORM:



SCHEMATIC DIAGRAM:



4. 4 – BIT BARREL SHIFTER

TOOL USED: VIVADO (XILINX).

CODE:

Design block

```
module barrel_shift(
input  [3:0]data_in,     // 4-bit input data
input  [1:0]shift_amt,   // Shift amount (0 to 3)
input  dir,         // Direction: 0 = Shift Right, 1 = Shift Left
output [3:0]data_out     // 4-bit shifted output
    );

   assign data_out = dir ? (data_in << shift_amt) : (data_in >> shift_amt);
endmodule
```

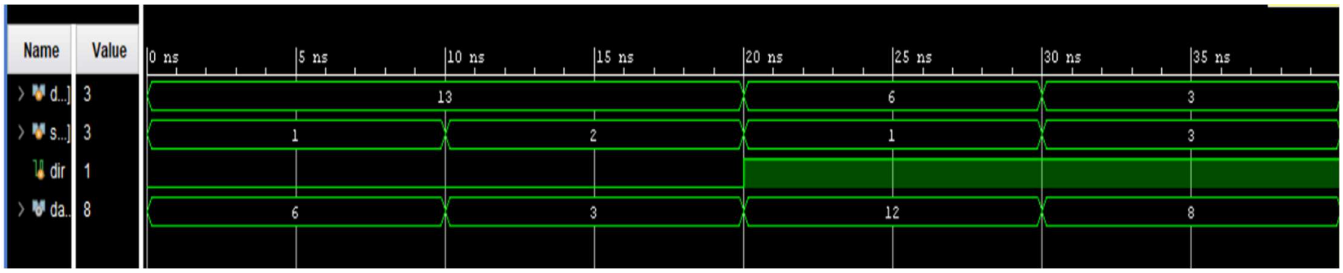
Test bench

```
module tb_barrel_shift;
reg  [3:0]data_in;
reg  [1:0]shift_amt;
reg  dir;
wire [3:0]data_out;
barrel_shift B1 (data_in,shift_amt,dir,data_out);
initial begin
$monitor("Time =%0t  direction =%b  shifted amount =%d  data in =%b  data out =%b",
                 $time, dir, shift_amt, data_in, data_out);
// Test 1: Shift Right by 1 (dir = 0)
data_in = 4'b1101; dir = 1'b0; shift_amt = 2'd1; #10;
//Test 2: Shift Right by 2
data_in = 4'b1101; dir = 1'b0; shift_amt = 2'd2; #10;
//Test 3: Shift Left by 1 (dir = 1)
data_in = 4'b0110; dir = 1'b1; shift_amt = 2'd1; #10;
//Test 4: Shift Left by 3
data_in = 4'b0011; dir = 1'b1; shift_amt = 2'd3; #10;
$finish;
end
endmodule
```

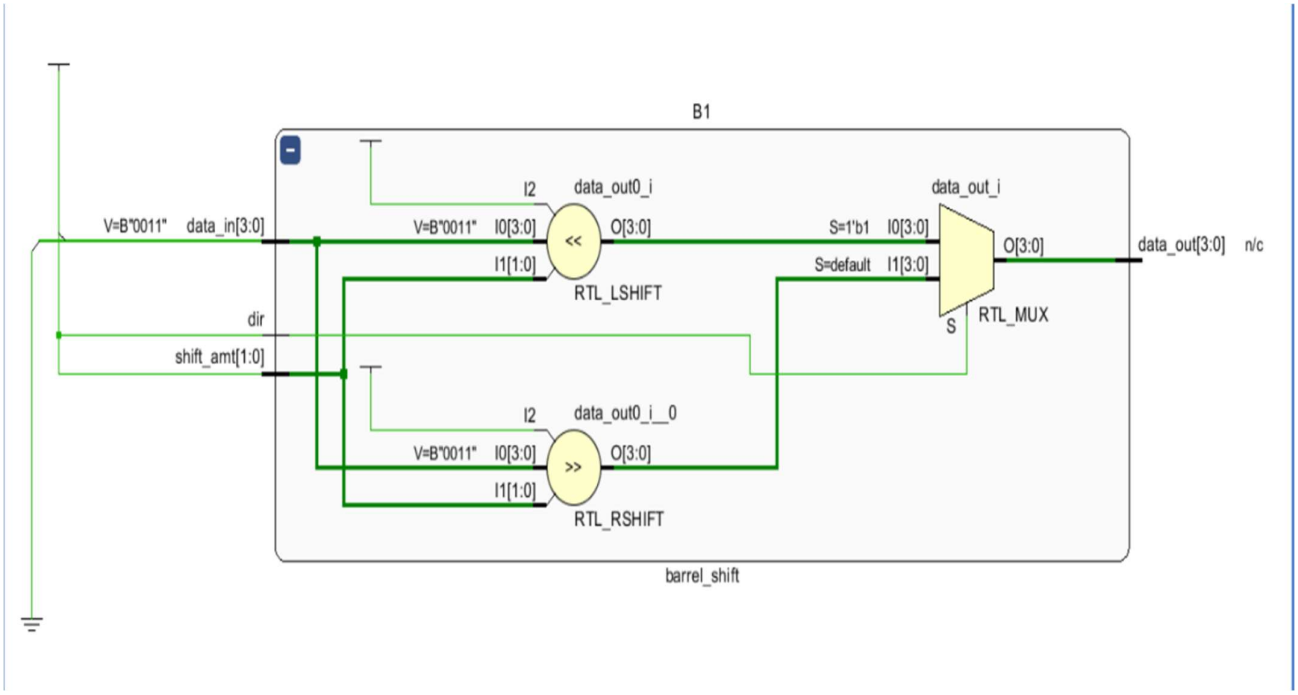
OUTPUT:

```
# run 1000ns
Time =0  direction =0  shifted amount =1  data in =1101  data out =0110
Time =10000  direction =0  shifted amount =2  data in =1101  data out =0011
Time =20000  direction =1  shifted amount =1  data in =0110  data out =1100
Time =30000  direction =1  shifted amount =3  data in =0011  data out =1000
```

WAVEFORM:



SCHEMATIC DIAGRAM:



5. TIMER MODULE

TOOL USED: VIVADO (XILINX).

CODE:

Design block

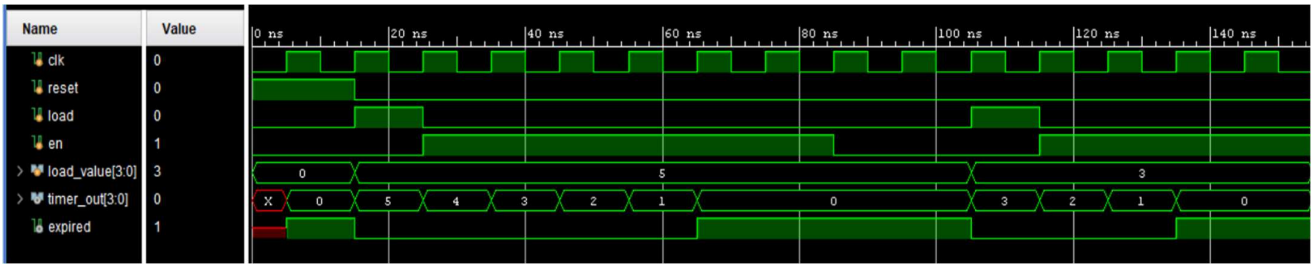
```
module timer (
    input clk,
    input reset,
    input load,         // Load new value into timer
    input en,
    input [3:0] load_value,   // Starting value for the countdown
    output reg [3:0] timer_out,// Current countdown value
    output expired       // High (1) when timer reaches 0
);
    assign expired = (timer_out == 4'b0000);

    always @(posedge clk) begin
        if (reset) begin
            timer_out <= 4'b0000;
        end
        else if (load) begin
            timer_out <= load_value; // Load initial value
        end
        else if (en && !expired) begin
            timer_out <= timer_out - 1'b1; // Count down
        end
    end
endmodule
```

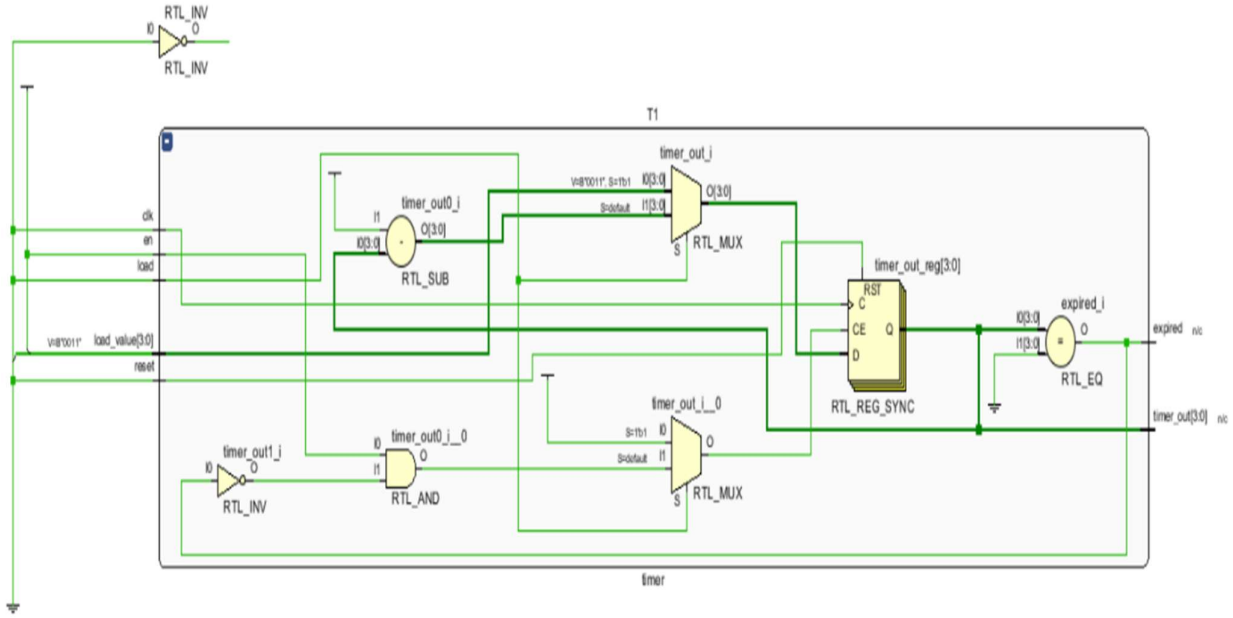
Test bench

```
module tb_timer;
reg clk;
reg reset;
reg load;
reg en;
reg [3:0] load_value;
wire [3:0] timer_out;
wire expired;
timer T1 (clk,reset,load,en,load_value,timer_out,expired);
always #5 clk=~clk;
initial begin
clk = 0; reset = 1; load = 0; en = 0; load_value = 4'd0;#15
reset = 0;
// Load value 5 into the timer
load_value = 4'd5;load = 1;#10
load = 0;en = 1;#60
// Pause counting
en = 0;#20
// Load a new value (3) and resume
load_value = 4'd3;load = 1; #10
load = 0; en = 1;#40
$finish;
end
endmodule
```

OUTPUT WAVEFORM:



SCHEMATIC DIAGRAM:



6. RING COUNTER

TOOL USED: VIVADO(XILINX).

CODE:

Design block

```
module ring_counter (
    input clk, reset,
    output reg [3:0] q
);
    always @(posedge clk) begin
        if (reset) begin
            q <= 4'b0001;
        end
        else begin
            q <= {q[2:0], q[3]};
        end
    end
endmodule
```

Test bench

```
`timescale 1ns / 1ps

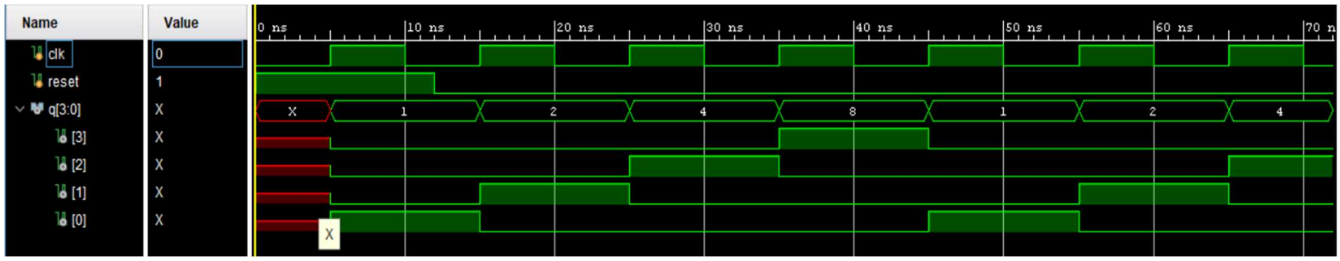
module tb_ring_center;

    reg        clk;
    reg        reset;
    wire [3:0] q;
    ring_counter R1 (clk,reset,q);
    always #5 clk = ~clk;
    initial begin
        clk = 0;
        reset = 1;#12
        reset = 0;#60
        $finish;
    end
    initial begin
        $monitor("Time=%0t | Reset=%b | Q = %b", $time, reset, q);
    end
endmodule
```

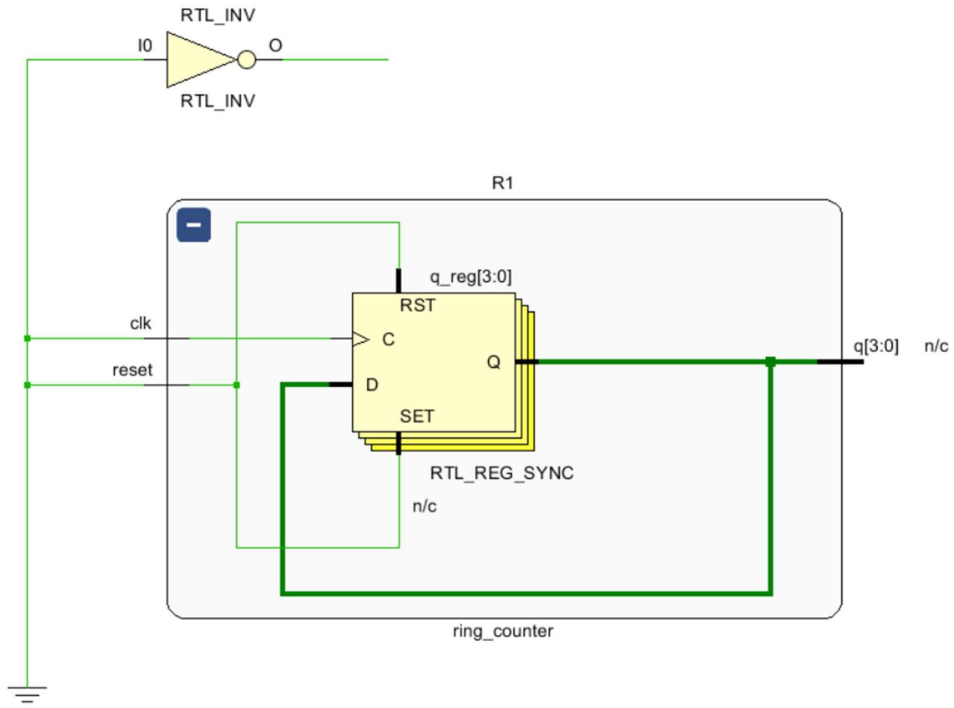
OUTPUT:

```
# run 1000ns
Time=0 | Reset=1 | Q = xxxx
Time=5000 | Reset=1 | Q = 0001
Time=12000 | Reset=0 | Q = 0001
Time=15000 | Reset=0 | Q = 0010
Time=25000 | Reset=0 | Q = 0100
Time=35000 | Reset=0 | Q = 1000
Time=45000 | Reset=0 | Q = 0001
Time=55000 | Reset=0 | Q = 0010
Time=65000 | Reset=0 | Q = 0100
```

WAVEFORM:



SCHEMATIC DIAGRAM:



7. JOHNSON COUNTER

TOOL USED: VIVADO(XILINX).

CODE:

Design block

```
module johnson_counter (
    input clk,
    input reset,
    output reg [3:0] q );

    always @(posedge clk) begin
        if (reset) begin
            q <= 4'b0000;
        end
        else begin
            q <= {q[2:0], ~q[3]};
        end
    end
endmodule
```

Test bench

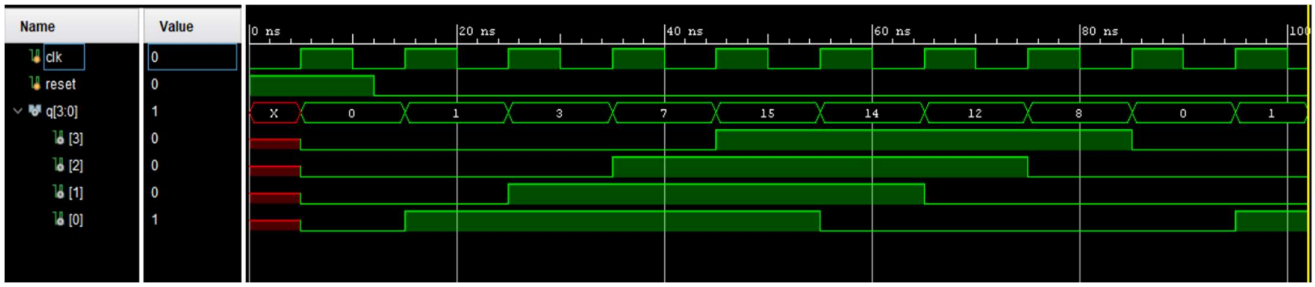
```
`timescale 1ns / 1ps

module tb_johnson_counter;
    reg clk;
    reg reset;
    wire [3:0] q;
    johnson_counter J1 (clk,reset,q);
    always #5 clk = ~clk;
    initial begin
        clk = 0;reset = 1;#12
        reset = 0;#90
        $finish;
    end
    initial begin
        $monitor("Time=%0t | Reset=%b | Q = %b", $time, reset, q);
    end
endmodule
```

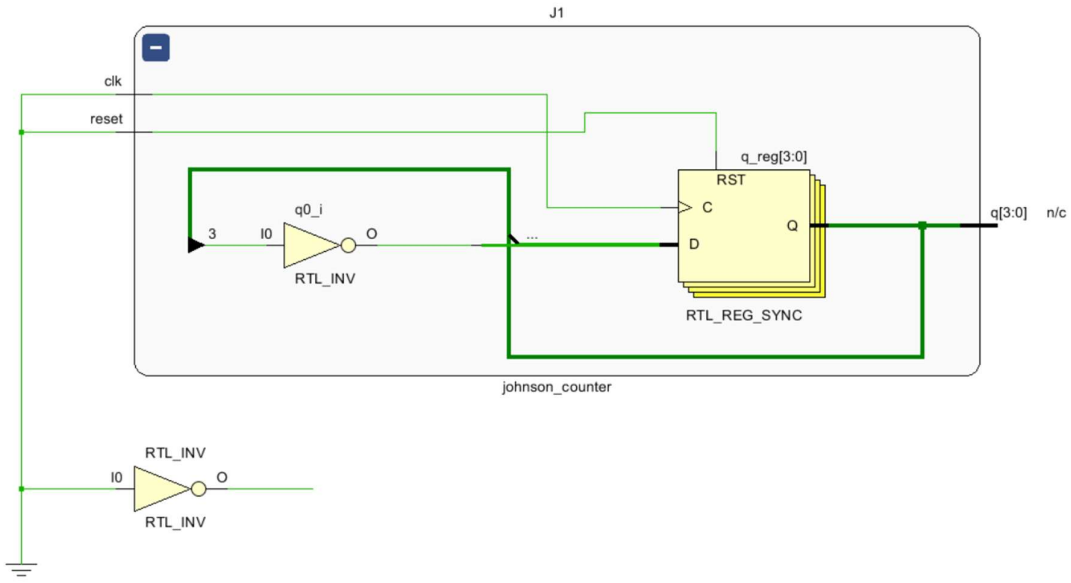
OUTPUT:

```
# run 1000ns
Time=0 | Reset=1 | Q = xxxx
Time=5000 | Reset=1 | Q = 0000
Time=12000 | Reset=0 | Q = 0000
Time=15000 | Reset=0 | Q = 0001
Time=25000 | Reset=0 | Q = 0011
Time=35000 | Reset=0 | Q = 0111
Time=45000 | Reset=0 | Q = 1111
Time=55000 | Reset=0 | Q = 1110
Time=65000 | Reset=0 | Q = 1100
Time=75000 | Reset=0 | Q = 1000
Time=85000 | Reset=0 | Q = 0000
Time=95000 | Reset=0 | Q = 0001
```

WAVEFORM:



SCHEMATIC DIAGRAM:



8. SYNCHRONOUS, ASYNCHRONOUS COUNTERS

TOOL USED: VIVADO(XILINX).

- **ASYNCHRONOUS COUNTER**

CODE:

Design block

```
module syn_count(
    input clk,reset,
    output reg [3:0] q
    );
    always @ (posedge clk)
    begin
    if (reset)
    q = 4'b0000;
    else
    q=q+1;
    end
endmodule
```

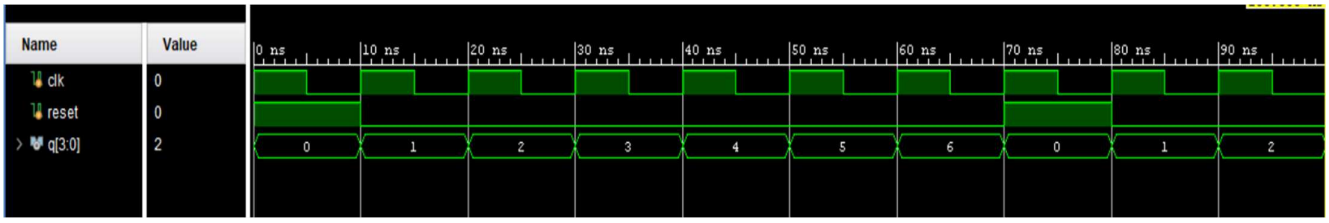
Test bench

```
module syn_count_tb;
reg clk,reset;
wire [3:0] q;
syn_count scl (clk,reset,q);
always #5 clk=~clk;
initial begin
$monitor("Time=%0t | Reset=%b | Q = %b", $time, reset, q);
clk =1; reset =1;#10
reset=0;#60
reset=1;#10
reset=0;#20
$finish;
end
endmodule
```

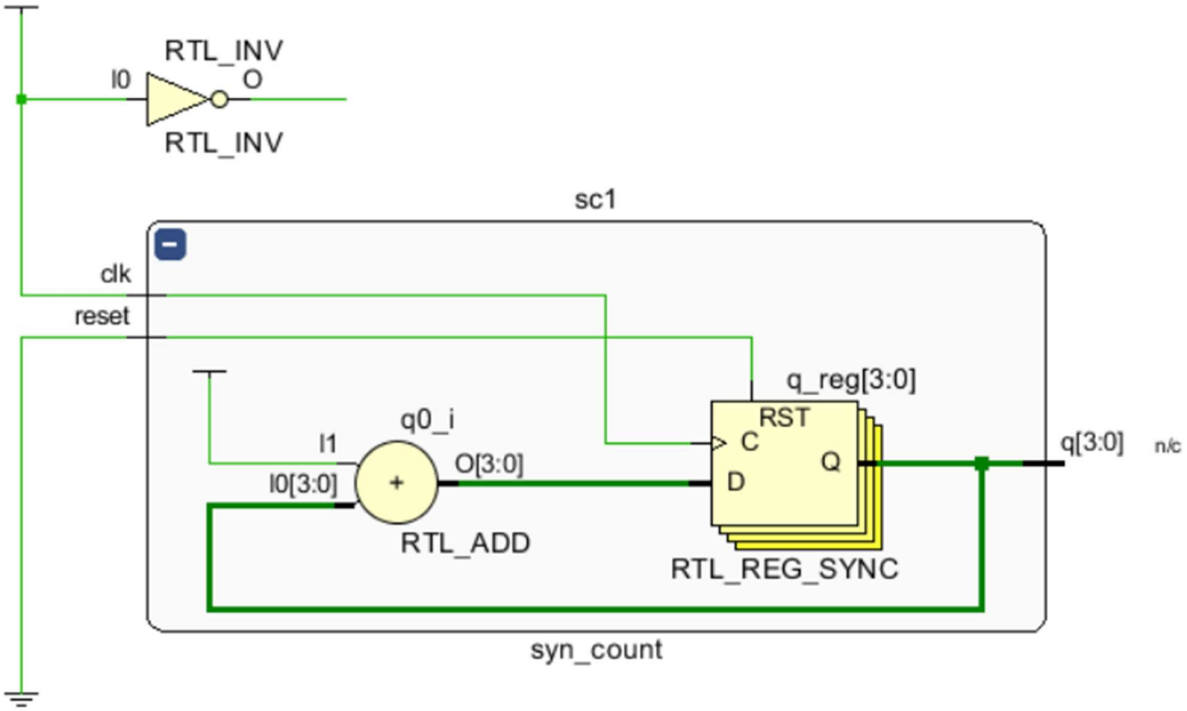
OUTPUT:

```
# run 1000ns
Time=0 | Reset=1 | Q = 0000
Time=10000 | Reset=0 | Q = 0001
Time=20000 | Reset=0 | Q = 0010
Time=30000 | Reset=0 | Q = 0011
Time=40000 | Reset=0 | Q = 0100
Time=50000 | Reset=0 | Q = 0101
Time=60000 | Reset=0 | Q = 0110
Time=70000 | Reset=1 | Q = 0000
Time=80000 | Reset=0 | Q = 0001
Time=90000 | Reset=0 | Q = 0010
```

WAVEFORM:



SCHEMATIC DIAGRAM:



- **ASYNCHRONOUS COUNTER**

CODE:

Deign block

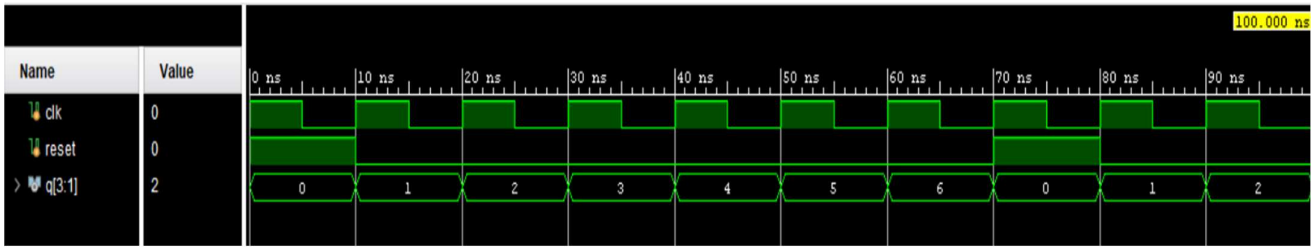
```
module async_count(
input clk,reset,
output [3:1] q
    );
    //up counter(0-6)
     jk_ff jk4 (clk,1,1,reset,q[1]);
     jk_ff jk5 (~q[1],1,1,reset,q[2]);
     jk_ff jk6 (~q[2],1,1,reset,q[3]);
endmodule

module jk_ff (
input clk,j,k,reset,
output reg q);
always @ (posedge clk or posedge reset)
begin
if (reset)
 q=0;
else
case ({j,k})
2'b00:q=q;
2'b01:q=0;
2'b10:q=1;
2'b11:q=~q;
endcase
end
endmodule
```

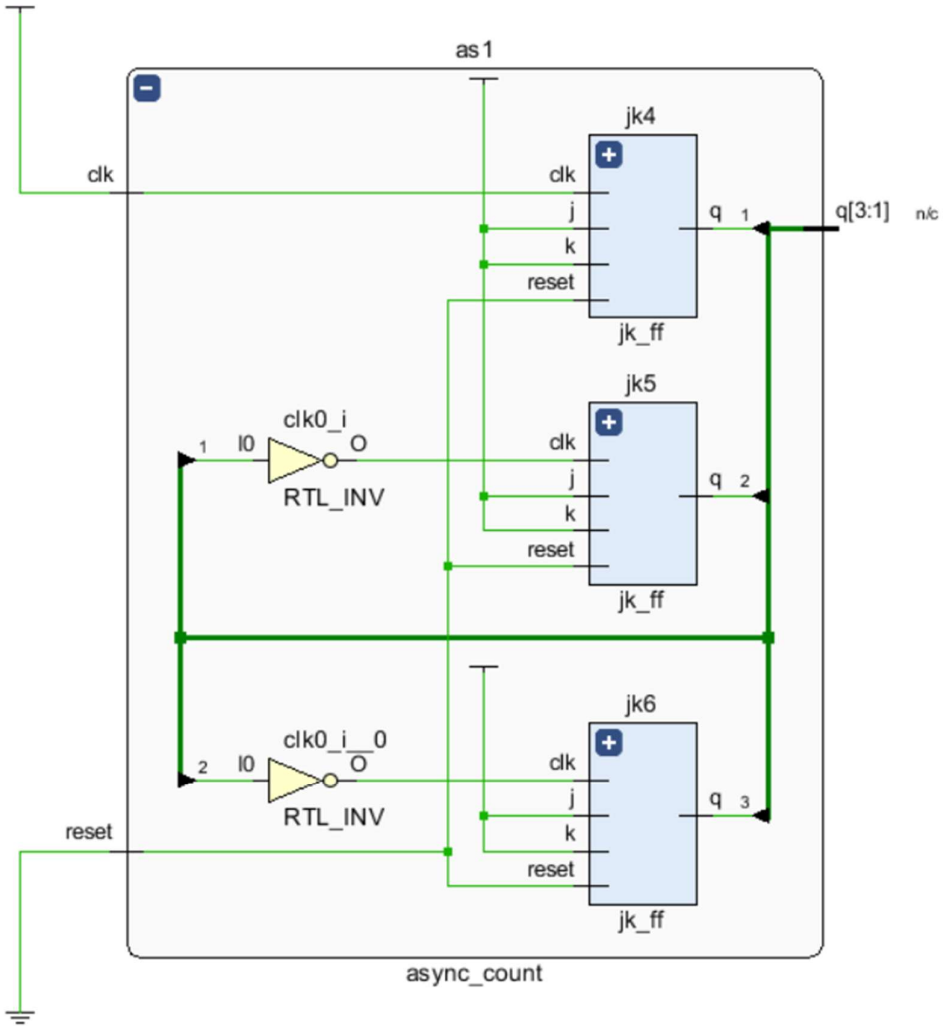
Test bench

```
module async_count_tb;
reg clk,reset;
wire [3:1]q;
async_count as1 (clk,reset,q);
initial
forever #5 clk = ~clk;
initial begin
clk =1;reset=1;#10
reset=0;#60
reset=1;#10
reset=0;#20
$finish;
end
endmodule
```

OUTPUT WAVEFORM:



SCHEMATIC DIAGRAM:



9. SHIFT REGISTER (SIPO)

TOOL USED: VIVADO(XILINX).

CODE:

Design block

```
module shift_reg_sipo (
    input clk,
    input reset,
    input serial_in,
    output reg [3:0] q
);
    always @(posedge clk) begin
        if (reset) begin
            q <= 4'b0000;
        end else begin
            q <= {serial_in, q[3:1]};
        end
    end
endmodule
```

Test bench

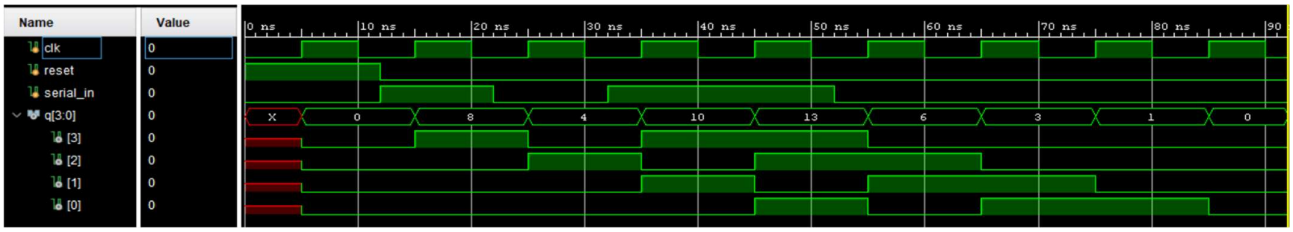
```
`timescale 1ns / 1ps

module tb_shift_regt;
reg clk;
reg reset;
reg serial_in;
wire [3:0] q;
shift_reg_sipo S1 (clk,reset,serial_in,q);
always #5 clk = ~clk;
initial begin
    clk = 0;reset = 1;serial_in = 0;#12
    reset = 0;
    // Shift in a sequence of bits: 1, 0, 1, 1
    serial_in = 1'b1; #10;
    serial_in = 1'b0; #10;
    serial_in = 1'b1; #10;
    serial_in = 1'b1; #10;
    serial_in = 1'b0; #40;
    $finish;
end
initial begin
    $monitor("Time=%0t | Reset=%b | Serial_In=%b | Q=%b", $time, reset, serial_in, q);
end
endmodule
```

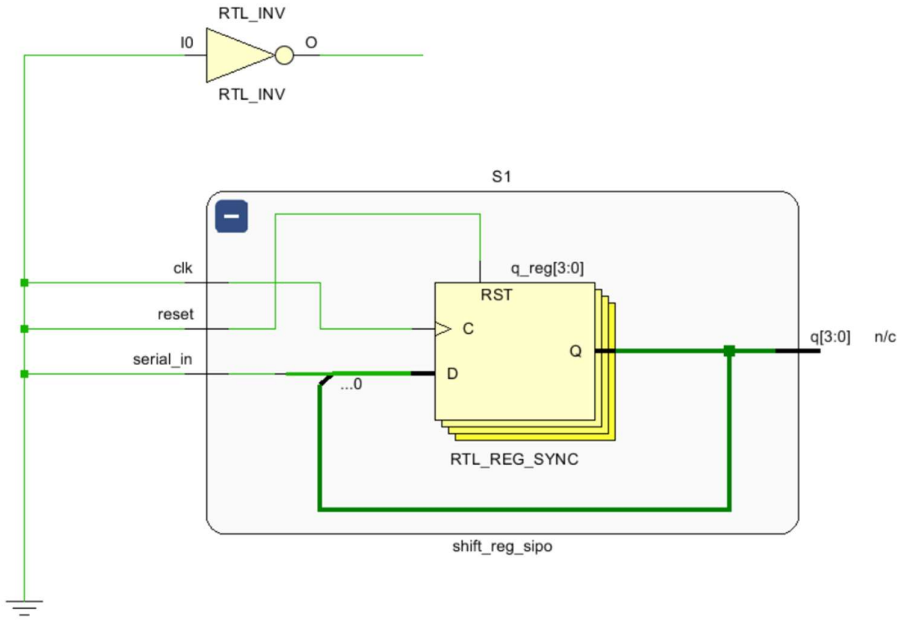
OUTPUT:

```
# run 1000ns
Time=0 | Reset=1 | Serial_In=0 | Q=xxxx
Time=5000 | Reset=1 | Serial_In=0 | Q=0000
Time=12000 | Reset=0 | Serial_In=1 | Q=0000
Time=15000 | Reset=0 | Serial_In=1 | Q=1000
Time=22000 | Reset=0 | Serial_In=0 | Q=1000
Time=25000 | Reset=0 | Serial_In=0 | Q=0100
Time=32000 | Reset=0 | Serial_In=1 | Q=0100
Time=35000 | Reset=0 | Serial_In=1 | Q=1010
Time=45000 | Reset=0 | Serial_In=1 | Q=1101
Time=52000 | Reset=0 | Serial_In=0 | Q=1101
Time=55000 | Reset=0 | Serial_In=0 | Q=0110
Time=65000 | Reset=0 | Serial_In=0 | Q=0011
Time=75000 | Reset=0 | Serial_In=0 | Q=0001
Time=85000 | Reset=0 | Serial_In=0 | Q=0000
```

WAVEFORM:



SCHEMATIC DIAGRAM:



10. DIVIDE BY N COUNTER

TOOL USED: VIVADO (XILINX).

CODE:

Design block

```
module divide_by_n #(parameter N = 5 )(
    input        clk,
    input        reset,
    output reg   div_out); // Divided output pulse
    // Automatically calculate the number of bits required for count (0 to N-1)
    reg [$clog2(N)-1:0] count;
    always @(posedge clk) begin
        if (reset) begin
            count <= 0;
            div_out <= 0;
        end
        else if (count == N - 1) begin
            count <= 0;
            div_out <= 1'b1;
        end
        else begin
            count <= count + 1'b1;
            div_out <= 1'b0;
        end
    end
endmodule
```

Test bench

```
`timescale 1ns / 1ps

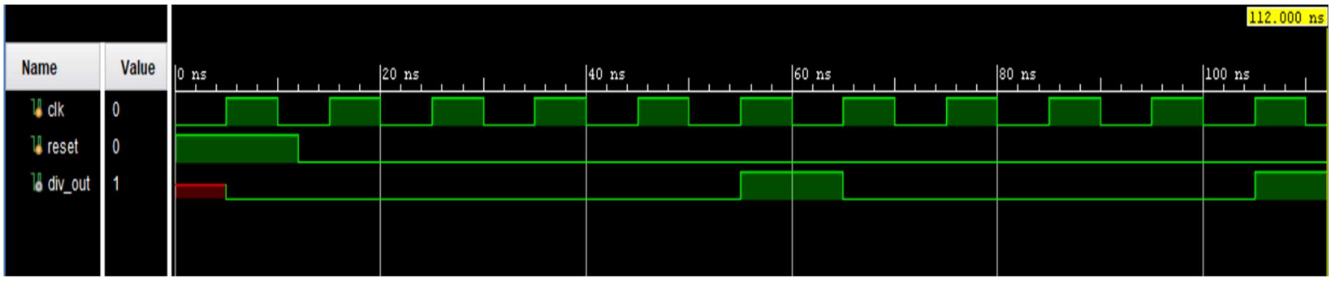
module tb_divide_by_n;
    reg        clk;
    reg        reset;
    wire       div_out;

    divide_by_n N1 (clk, reset, div_out);
    always #5 clk = ~clk;
    initial begin
        clk = 0;reset = 1;#12
        reset = 0;#100
        $finish;
    end
    initial begin
        $monitor("Time=%0t | Reset=%b | Div_Out=%b", $time, reset, div_out);
    end
endmodule
```

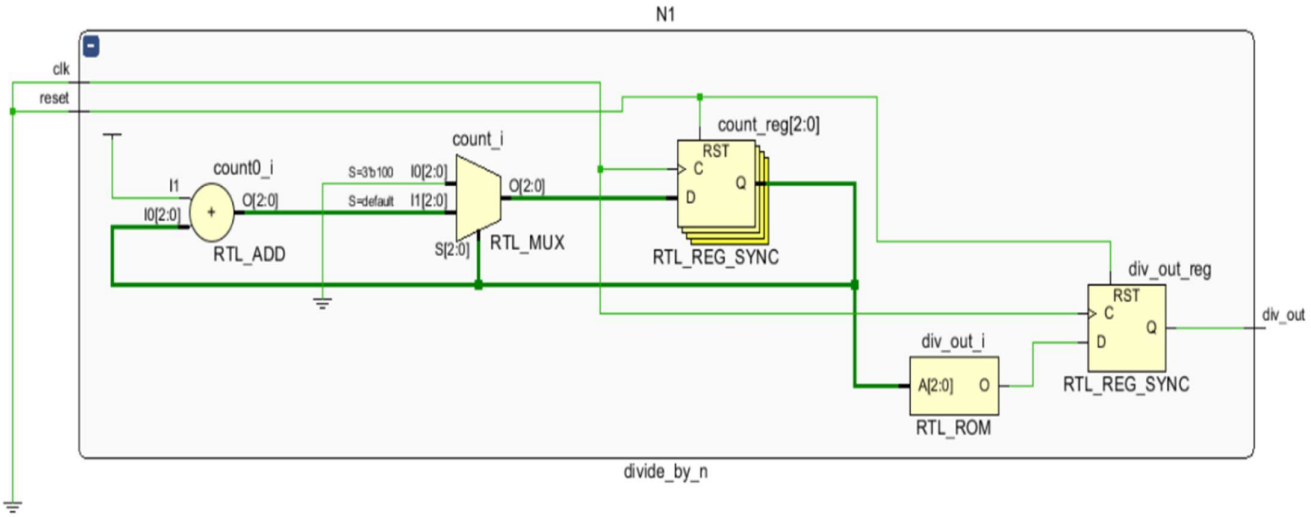
OUTPUT:

```
# run 1000ns
Time=0 | Reset=1 | Div_Out=x
Time=5000 | Reset=1 | Div_Out=0
Time=12000 | Reset=0 | Div_Out=0
Time=55000 | Reset=0 | Div_Out=1
Time=65000 | Reset=0 | Div_Out=0
Time=105000 | Reset=0 | Div_Out=1
```

WAVEFORM:



SCHEMATIC DIAGRAM:



11. MEALY AND MOORE FSM

TOOL USED: VIVADO(XILINX).

- **MEALY SEQUENCE DETECTOR**

CODE:

Design block

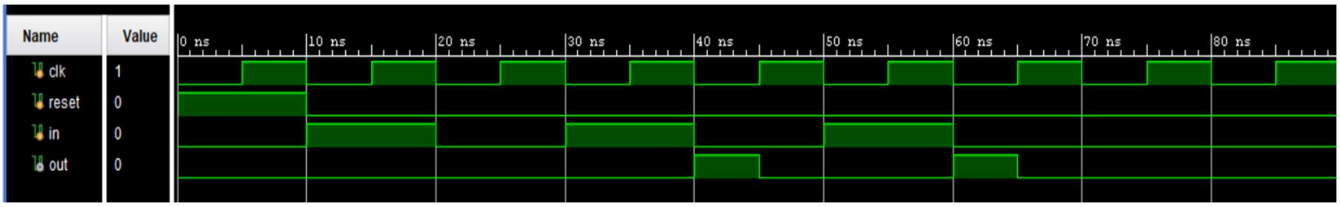
```
// 1010 mealy (overlapping) sequence detector

module seq_detect(
input clk, reset, in,
output reg out
    );
    reg [1:0]ps,ns;
    parameter s0=2'b00, s1=2'b01, s2=2'b10, s3=2'b11;
    always @ (ps,in)
    begin
    out =0;
    case (ps)
    s0:ns=(in?s1:s0);
    s1:ns=(in?s1:s2);
    s2:ns=(in?s3:s0);
    s3:begin
    ns=(in?s1:s2);
    out=(in?0:1);
    end
    default:ns=s0;
    endcase
    end
    always @(posedge clk or posedge reset) begin
    if (reset)
    ps<=s0;
    else
    ps<=ns;
    end
endmodule
```

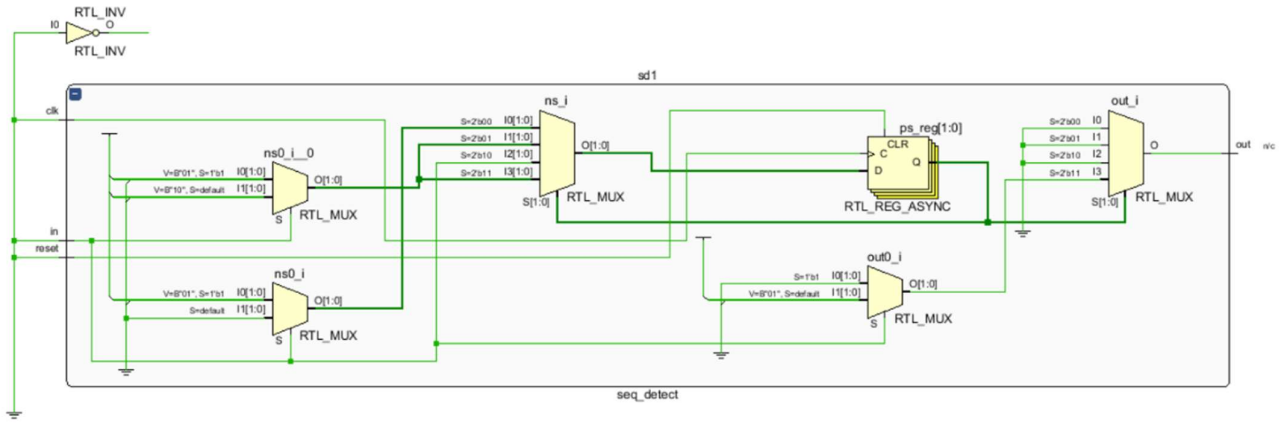
Test bench

```
module tb;
reg clk , reset , in;
wire out;
seq_detect sd1 (clk, reset,in, out);
initial begin
clk=0;reset=1; in=0;#10;
reset=0;in=1;#10
in=0;#10
in=1;#10
in=0;#10
in=1;#10
in=0;#10
in=0;#20
$finish;
end
always #5 clk=~clk;
endmodule
```

OUTPUT WAVEFORM:



SCHEMATIC DIAGRAM:



- **MOORE SEQUENCE DETECTOR**

CODE:

Deign block

```
//detect 1011
module moore_seq_detect(
    input clk, reset, in,
    output reg out
    );
reg [2:0] ps, ns;
parameter s0=3'b000, s1=3'b001, s2=3'b010, s3=3'b011, s4=3'b100;
always @ (ps, in)
begin
out=0;
case (ps)
 s0:ns=in?s1:s0;
 s1:ns=in?s1:s2;
 s2:ns=in?s3:s0;
 s3:ns=in?s4:s2;
 s4:begin
  ns=in?s1:s2;
  out=1;
  end
  default: begin
            ns=s0;
            end
endcase
end
always @ (posedge clk)
begin
 if (reset) ps<=0;
 else ps<=ns;
end
endmodule
```

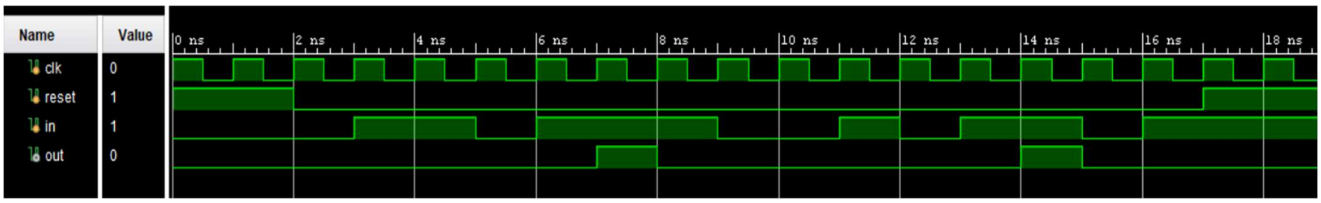
Test bench

```
`timescale 1ns / 1ps

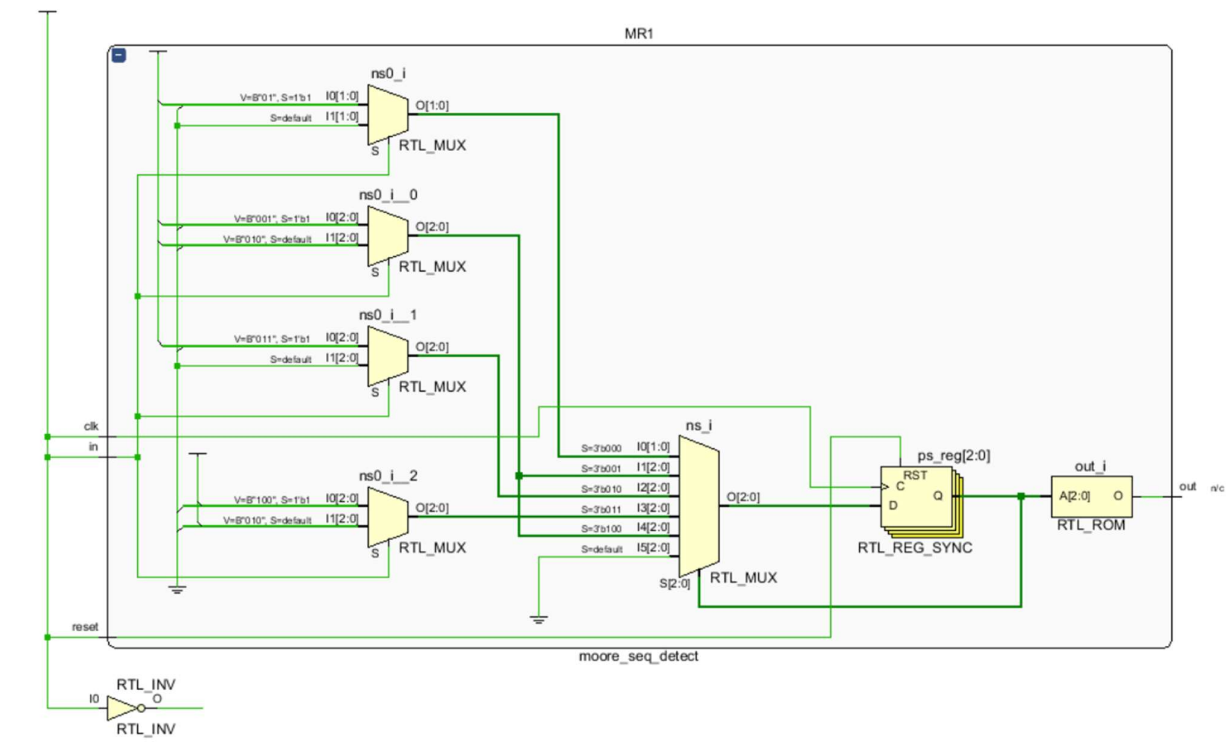
module moore_tb;
reg clk, reset, in;
wire out;
moore_seq_detect MR1 (clk,reset,in,out);
initial begin
clk=1;reset=1;in=0;#2
reset=0;
in=0;#1
in=1;#1
in=1;#1
in=0;#1
in=1;#1
in=1;#1
```

```
in=1;#1
in=0;#1
in=0;#1
in=1;#1
in=0;#1
in=1;#1
in=1;#1
in=0;#1
in=1;#1
reset=1;in=1;#2
$finish;
end
always #0.5 clk=~clk;
endmodule
```

OUTPUT WAVEFORM:



SCHEMATIC DIAGRAM



12. RANDOM SEQUENCE GENERATOR

TOOL USED: VIVADO (XILINX).

CODE:

Design block

```
module random_seq_generator (
    input        clk,
    input        reset,
    output reg [3:0] rand_out // 4-bit pseudo-random output
);
    // Feedback polynomial: x^4 + x^3 + 1
    wire feedback;
    assign feedback = rand_out[3] ^ rand_out[2];
    always @(posedge clk) begin
        if (reset) begin
            rand_out <= 4'b0001;
        end
        else begin
            // Shift left and insert the feedback bit into the LSB position
            rand_out <= {rand_out[2:0], feedback};
        end
    end
endmodule
```

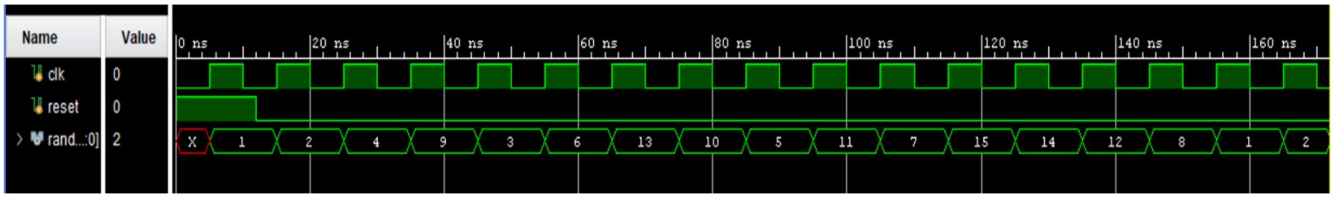
Test bench

```
module tb_random_seq_generator;
    reg clk;
    reg reset;
    wire [3:0] rand_out;
    random_seq_generator R1 (clk,reset,rand_out);
    always #5 clk = ~clk;
    initial begin
        clk = 0; reset = 1;#12
        reset = 0;#160
        $finish;
    end
    initial begin
        $monitor("Time=%0t | Reset=%b | Binary = %b | Decimal = %d",
                 $time, reset, rand_out, rand_out);
    end
endmodule
```

OUTPUT:

```
# run 1000ns
Time=0 | Reset=1 | Binary = xxxx | Decimal =  x
Time=5000 | Reset=1 | Binary = 0001 | Decimal =  1
Time=12000 | Reset=0 | Binary = 0001 | Decimal =  1
Time=15000 | Reset=0 | Binary = 0010 | Decimal =  2
Time=25000 | Reset=0 | Binary = 0100 | Decimal =  4
Time=35000 | Reset=0 | Binary = 1001 | Decimal =  9
Time=45000 | Reset=0 | Binary = 0011 | Decimal =  3
Time=55000 | Reset=0 | Binary = 0110 | Decimal =  6
Time=65000 | Reset=0 | Binary = 1101 | Decimal = 13
Time=75000 | Reset=0 | Binary = 1010 | Decimal = 10
Time=85000 | Reset=0 | Binary = 0101 | Decimal =  5
Time=95000 | Reset=0 | Binary = 1011 | Decimal = 11
Time=105000 | Reset=0 | Binary = 0111 | Decimal =  7
Time=115000 | Reset=0 | Binary = 1111 | Decimal = 15
Time=125000 | Reset=0 | Binary = 1110 | Decimal = 14
Time=135000 | Reset=0 | Binary = 1100 | Decimal = 12
Time=145000 | Reset=0 | Binary = 1000 | Decimal =  8
Time=155000 | Reset=0 | Binary = 0001 | Decimal =  1
Time=165000 | Reset=0 | Binary = 0010 | Decimal =  2
```

WAVEFORM:



SCHEMATIC DIAGRAM

